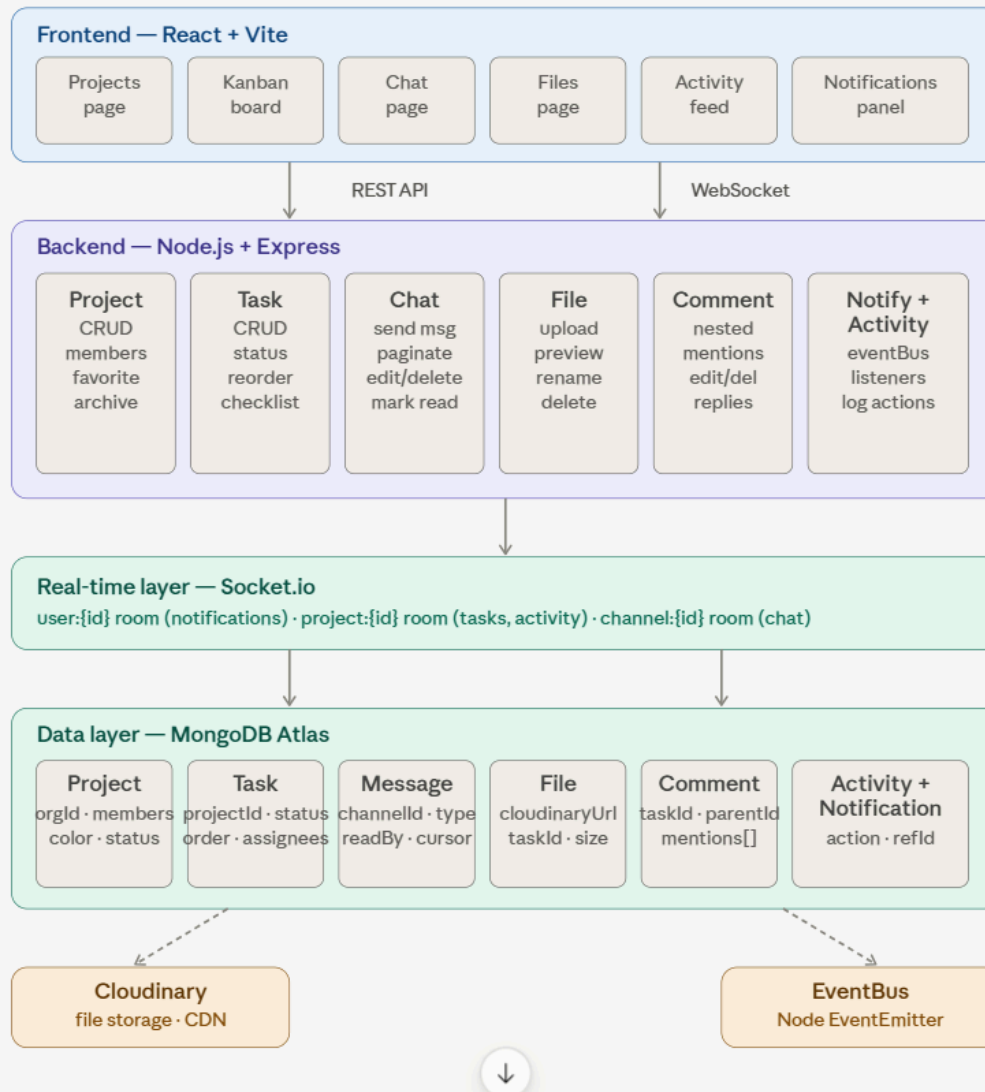
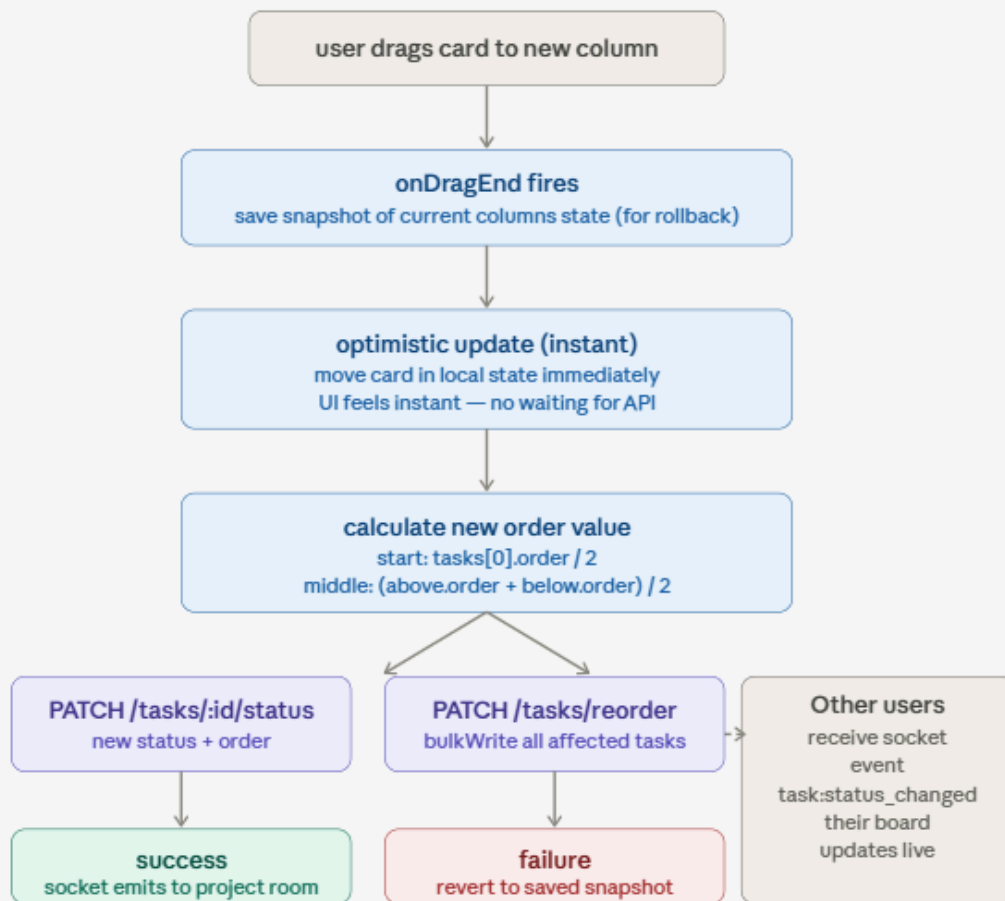


Phase 2 — complete system architecture

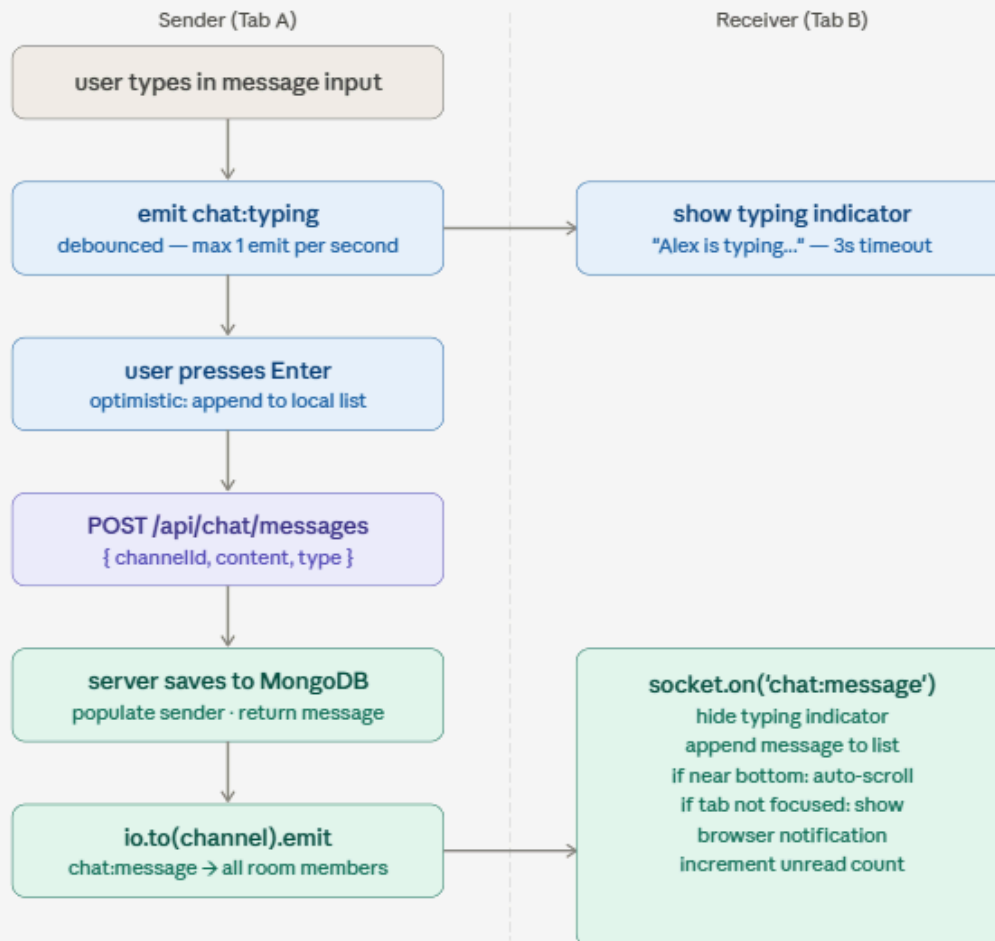


Flow 1 — Drag and drop + task reorder



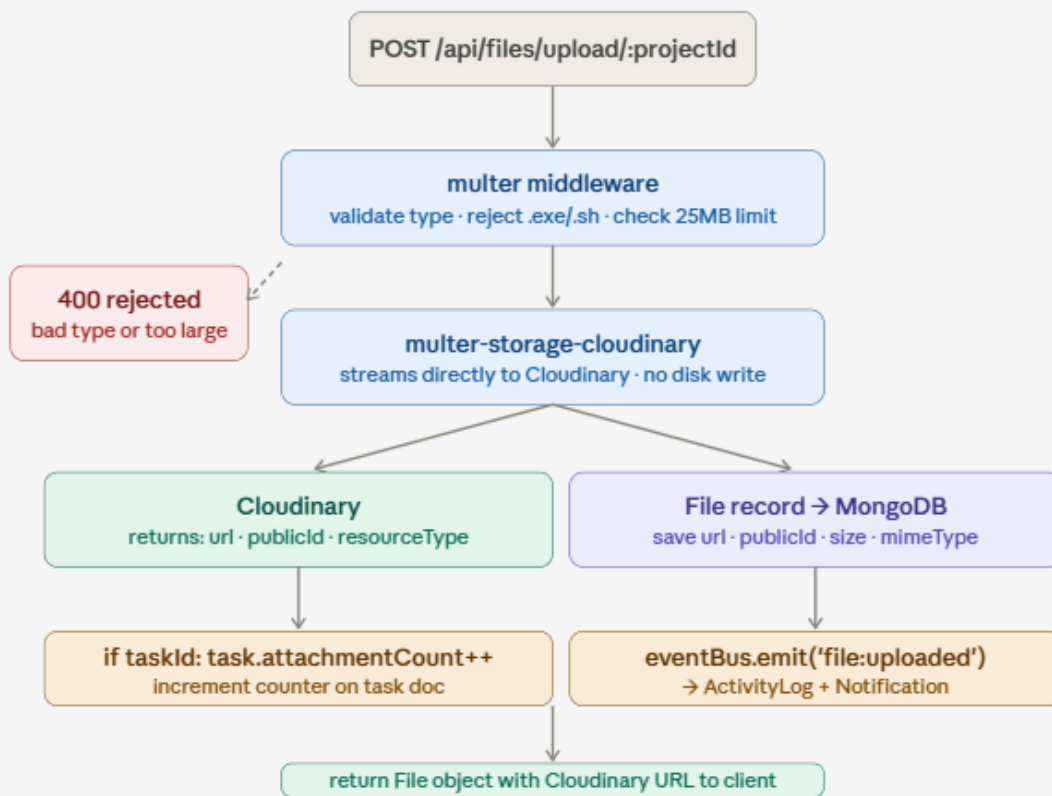
Interview explanation: "Optimistic updates are the key design decision here. When a user drags a card, I update the UI immediately — no waiting. I save a snapshot of the board state before the update. I fire two API calls in parallel: one to update the task's status, one to persist the new order via `bulkWrite` — a single MongoDB round trip for multiple updates. If either fails, I restore the snapshot. The user never sees a flash or jump. Meanwhile the backend emits a socket event to the project room so every other open browser tab sees the change instantly."

Flow 2 — Real-time chat + typing indicator



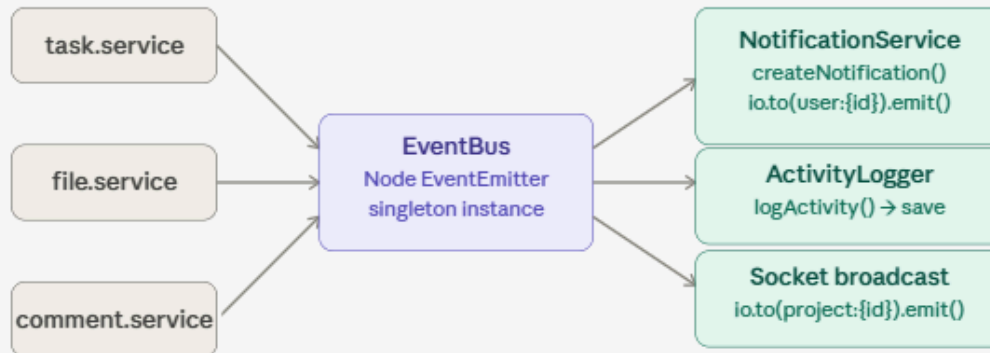
Interview explanation: "Messages go through REST — not pure socket — for two reasons: persistence and reliability. A socket emit has no delivery guarantee; a REST call creates the DB record first, then the server broadcasts. The typing indicator is pure socket with a debounce — only 1 emit per second per user. On the receiver side I set a 3-second timeout: if no new typing event arrives, the indicator hides. This prevents 'stuck' typing indicators when someone closes the tab mid-sentence."

Flow 3 — File upload pipeline



Interview explanation: "Files never touch the server disk — `multer-storage-cloudinary` streams the upload directly from the incoming request to Cloudinary. My server is stateless. Cloudinary returns the URL and a `publicId`. I save both — the URL for display, the `publicId` for deletion (Cloudinary needs it to destroy the asset). After saving I emit an event on the `eventBus` which the `ActivityLog` and `Notification` listeners pick up independently — adding a new side effect means adding a new listener, not touching the upload code."

Flow 4 — EventBus + notification + activity log



Event → listener mapping (eventListeners.js)

```
task:created      → notify each assignee (except creator) · log 'task.created'
task:status_changed → if done: notify creator · log 'task.status_changed'
file:uploaded     → log 'file.uploaded' · emit to project room
comment:created   → notify task assignees · log 'task.commented'
user:mentioned    → notify mentioned user with 'mention' type
```

Interview explanation: "This is the Observer pattern. The `eventBus` is a Node.js `EventEmitter` singleton. Every service just emits — it has no idea who's listening. The `eventListeners.js` file registers all subscribers at startup. This means adding a new side effect — say, sending a Slack webhook when a task is created — means adding one new listener. Nothing else changes. It also means the notification and activity log run in parallel, not sequentially. This is the Open/Closed principle from SOLID applied directly."

Phase 2 — complete checklist

WEEK 3 — PROJECTS + KANBAN + TASKS

...

✔ Project CRUD

create · update · archive · delete · favorite · color picker

✔ Task model + CRUD

status pipeline · priority · assignees · labels · order field

✔ Kanban board UI

5 columns · @hello-pangea/dnd · column colors · card design

✔ Drag and drop

optimistic update · fractional order · snapshot rollback

✔ Task detail panel

slide-in · inline edit · checklist · assignees · status select

✔ Reorder API

bulkWrite · single DB round trip · fractal position indexing

✔ Projects page

grid · filter pills · project card · color bar · progress bar

✔ EventBus wired

eventBus.js created · emits in task service · ready for listeners

WEEK 4 — REAL-TIME CHAT + NOTIFICATIONS

✔ Socket.io setup

JWT auth middleware · 3 room types · join/leave events

✔ Message model + API

cursor pagination · readBy array · edit/delete · channelId

✔ Chat page UI

channel list · DM list · online status · message grouping

✔ Typing indicator

debounce 1s emit · 3s receiver timeout · animated dots

✔ Notification system

model · service · 8 types · unread count · mark all read

✔ Notification panel UI

slide-in · real-time append · badge in sidebar · navigate on click

✔ Kanban real-time sync

socket listeners in TaskContext · other users see drops live

✔ SocketContext

auth token handshake · reconnect · cleanup on unmount

WEEK 5 — FILES + COMMENTS + ACTIVITY

✔ File upload pipeline

multer → Cloudinary stream · no disk write · publicId stored

✔ Files page UI

drag-drop zone · progress bar · file grid · type icons · preview

✔ Comment system

nested replies · @mentions · edit/delete · socket sync

✔ Activity log

9 action types · date grouping · real-time append · entity links

✔ EventBus listeners

all events wired · parallel notification + activity · OCP pattern

✔ Task detail — complete

attachments section + comments section fully wired

DESIGN SYSTEM + UI INFRASTRUCTURE

✔ AppLayout + TopBar + Sidebar

exact pixel spec · responsive collapse · breadcrumb · profile dropdown

✔ Shared components

Modal · SlidePanel · Avatar · Badge · Skeleton · EmptyState · ConfirmDialog

✔ Toast system

ToastContext · 4 variants · slide from right · 4s auto dismiss

✔ Error boundaries

KanbanPage · ChatPage · FilesPage each wrapped

The 12 questions interviewers will ask about Phase 2

"How does your Kanban board handle two users moving the same card simultaneously?"

Last write wins — whichever PATCH reaches the server last sets the final state. The other user's board receives a `task:status_changed` socket event and their state updates to match. I could add optimistic locking (version field + 409 on mismatch) as a future improvement — I'd mention that as a known tradeoff.

"Why use fractional indexing for task order instead of integers 1,2,3?"

Integer ordering requires updating every task after the insertion point. If I insert between positions 3 and 4, I'd need to shift every task from 4 onward. With fractional indexing I insert at 3.5 — only one write. Eventually floats lose precision so I periodically normalize back to integers with gaps (1000, 2000, 3000), but that's a background job.

"Why does chat use REST + socket instead of pure WebSocket?"

REST gives me persistence and delivery guarantees — the message is saved to MongoDB before anyone sees it. If the socket broadcast fails, the message still exists and loads on refresh. Pure socket is fire-and-forget. The REST call also gives me a proper HTTP response with the saved message object including its `_id` and timestamp.

"How does cursor-based pagination work for messages?"

Instead of `?page=3&limit=30` (offset), I use `?cursor=lastMessageId&limit=30`. The query is `Message.find({ channelId, _id: { $lt: cursor } }).sort({ _id: -1 }).limit(30)`. This is stable — if new messages arrive while the user scrolls back, the cursor position doesn't shift. Offset pagination has a classic bug where new messages push everything down and you skip or repeat items.

"How does your Socket.io room strategy work?"

Three room types: `user:{id}` for personal notifications, `project:{id}` for task and activity events, `channel:{id}` for chat messages. On connection, the server joins the socket to all of the user's relevant rooms by querying their project memberships. They only receive events for projects they belong to. Leaving a project removes them from that room — RBAC enforced at the socket level, not just REST.

"How do you handle the typing indicator — what if someone closes the tab?"

The typing indicator has a 3-second timeout on the receiver side. If no new `chat:typing` event arrives within 3 seconds, the indicator hides. Socket disconnect also fires `chat:stop_typing` from the server side via the `disconnect` event. So worst case the indicator shows for 3 seconds after the tab closes.

"Files never touch your server disk — how?"

`multer-storage-cloudinary` replaces multer's default disk storage. It intercepts the incoming multipart stream and pipes it directly to Cloudinary's upload API. My Express process

just proxies bytes — it never writes a temp file. This keeps the server stateless, which matters for horizontal scaling.

"How do nested comments work in the DB?"

I store `parentId: ObjectId | null` on each Comment. Top-level comments have `parentId: null`. When fetching, I get all comments for a task in one query, then group them client-side — `comments.filter(c => !c.parentId)` for top-level, then for each, `comments.filter(c => c.parentId === parent._id)` for replies. I limit visual nesting to 1 level (replies of replies are flat) to keep the UI clean.

"What is the EventBus and why not call notification service directly from task service?"

Direct calls create tight coupling — the task service would need to import and know about the notification service, the activity logger, and any future subscribers. With the EventBus (Node.js `EventEmitter`), the task service just emits an event. Each subscriber is registered independently in `eventListeners.js`. This is the Observer pattern. Adding a new side effect — like a webhook or an email — means one new listener, zero changes to task service. That's the Open/Closed principle.

"How do you prevent someone accessing another org's files?"

Every file has `orgId` and `projectId` stored (denormalized). Every API call for files goes through `authenticate` (verifies JWT) and then a membership check — the file service queries whether the requesting `userId` is in `project.members`. If not, `ApiError(403)`. The project ID in the route param is always verified against the authenticated user's memberships before any data is returned.

"What happens if Cloudinary is down during a file upload?"

The `multer-storage-cloudinary` call throws an error, which my global `errorHandler` catches and returns a 500. No file record is saved to MongoDB because the save only happens after Cloudinary responds. The client shows an error toast and the upload zone resets. I could add retry logic with exponential backoff as a future improvement.

"How does real-time activity feed work?"

When any action triggers `logActivity()`, after saving the `ActivityLog` document I emit `activity:new` to the `project:{id}` Socket.io room. Every open browser on that project receives it and prepends the item to the feed with a fade-in animation. The feed is date-grouped client-side. New items that arrive via socket include the populated `actorId` (name + avatar) so no additional fetch is needed.

The two data models to draw on a whiteboard

Task

```
_id, title, description
projectId → Project      (index)
orgId → Org              (denormalized, faster queries)
status: todo|in_progress|review|testing|done  (index with projectId)
priority: low|medium|high|urgent
assignees: [userId]      (index)
order: Number            (fractional, for column position)
labels: [String]
dueDate, estimatedHours
checklists: [{ id, text, done }]
attachmentCount, commentCount  (denormalized counters)
timestamps
```

Message

```
_id, content, type
channelId: String        (format: "project:{id}" or "dm:{id1}_{id2}")
senderId → User
readBy: [{ userId, readAt }]
isDeleted: Boolean       (soft delete – content replaced)
editedAt: Date
index: { channelId, createdAt } – cursor pagination
```

The `channelId` as a string (not `ObjectId`) is a deliberate design decision. It encodes both the type and the ID in one field. This lets me support both project channels and DMs in a single collection without a `type` discriminator field.